

TradeSaath — Full Tech Stack Explained (For a Complete Beginner)

This document lists **every major tool/service TradeSaath uses**, explains **what it actually is**, **why it was chosen**, **what it costs**, **what alternatives exist**, and **what could be done differently/better**. Written so someone with zero background can follow it.

1. Next.js (App Router) — The "Brain" of the Website

What it is (in plain words): Next.js is a framework (a pre-built toolkit) built on top of React (a JavaScript library for building user interfaces). Think of React as "LEGO blocks" for building web pages, and Next.js as the "instruction manual + extra parts" that make those blocks work as a real, fast, production website — with pages, navigation, server logic, and more, all in one project.

Why TradeSaath uses it:

- It lets one codebase handle both the **frontend** (what the user sees — dashboard, charts, buttons) and the **backend** (API routes that talk to the database, process trades, call AI) — so a solo founder doesn't need two separate projects.
- It has built-in support for **Server-Side Rendering (SSR)** — meaning pages can be built on the server before being sent to the browser, which is critical for SEO (Google can "read" the page) and for fast initial loads.
- "App Router" is the newest way Next.js organizes pages/folders — more powerful for mixing server and client logic.
- Huge community, tons of tutorials, and it's the industry-standard choice for solo/small-team SaaS founders right now.

What TradeSaath is currently missing because of this: Per the current build notes, tradesaath.com pages are largely **client-side rendered**, meaning the browser builds the page after loading — which leaves Google's crawler seeing a mostly empty page. The fix already identified is migrating key pages to **Server Components** (a core App Router feature) so content is baked in before it reaches the browser.

Alternatives and how they compare:

Alternative	What it is	Why not chosen / trade-off
Remix	Similar full-stack React framework, some say better data-loading model	Smaller community, fewer ready-made tutorials, less Vercel-native (deployment less seamless)
Plain React + Express backend	Two separate projects (frontend + backend)	More setup work, two things to deploy/maintain, no built-in SSR — worse for SEO
Vue/Nuxt	Different JS framework (Vue) with its own Next.js-equivalent (Nuxt)	Smaller job market, smaller ecosystem, no strong reason to switch given no prior Vue experience
SvelteKit	Newer, often praised for performance/simplicity	Smaller ecosystem, fewer AI-tooling integrations, riskier for a beginner-solo-founder to debug alone

Bottom line for a fresher: Next.js is the safe, well-documented, "everyone uses this" choice for a solo founder building a SaaS product fast. The main things to get right are learning the difference between **Server Components** (render on server, good for SEO/data) and **Client Components** (render in browser, needed for interactivity like buttons/forms).

2. Supabase — The Database

What it is: Supabase is a "Backend-as-a-Service." Underneath, it's a **PostgreSQL database** (a mature, reliable, industry-standard type of database that stores data in structured tables, like a very powerful Excel with relationships between sheets). Supabase wraps this database with a friendly dashboard, instant APIs, authentication tools, and file storage.

Why TradeSaath uses it:

- Free tier is generous enough for an early-stage product (fits the "limited budget, free-tier resources" approach).
- PostgreSQL is extremely reliable and battle-tested for financial/trading data (structured, relational — trades link to users link to sessions, etc.).
- Gives instant auto-generated APIs so you don't have to hand-write every single database query from scratch.
- Has **Row Level Security (RLS)** — a feature that lets you write rules directly into the database saying "user A can only see user A's trades" — critical for a multi-user financial app.

Alternatives:

Alternative	What it is	Trade-off
Firebase (Google)	NoSQL document database + backend services	Less suited to relational financial data (trades, sessions, KPIs are naturally relational/table-based); vendor lock-in to Google
PlanetScale (MySQL)	Serverless MySQL platform	No built-in RLS/auth/storage bundle like Supabase; would need separate auth service anyway
Raw AWS RDS + custom backend	Roll-your-own Postgres on AWS	Far more setup/maintenance overhead, not beginner-friendly, no free generous tier for early stage
MongoDB Atlas	NoSQL document database	Worse fit for structured, relational financial records; harder to enforce data consistency for things like KPI calculations

What could be better/watch out for:

- As TradeSaath scales, the free tier's database size/connection limits will need upgrading (paid tier).
- Database **connection pooling** becomes important once traffic grows — Supabase has a built-in "pooler" mode for this, worth learning early rather than late.

3. Clerk — User Login & Authentication

What it is: Clerk is a "plug-and-play" authentication service — it handles sign-up, login, password reset, social login (Google/GitHub sign-in), sessions, and security, so the founder doesn't have to build (and constantly patch) login systems from scratch.

Why TradeSaath uses it:

- Authentication is extremely easy to get subtly wrong (security holes), so using a dedicated, audited service is safer than DIY.
- Has ready-made, customizable UI components (login forms, user profile widgets) that drop straight into Next.js.
- Handles session/middleware logic that integrates cleanly with Next.js App Router.

Alternatives:

Alternative	What it is	Trade-off
Supabase Auth (built-in)	Authentication that comes free with Supabase itself	Would remove a whole separate service/bill — legitimate alternative; Clerk was likely chosen for nicer UI components and more control over user management flows, but this is one area where consolidating onto Supabase Auth alone could actually simplify the stack and cut cost
NextAuth.js / Auth.js	Free, open-source auth library for Next.js	More manual setup, more code to maintain yourself, but zero extra monthly cost
Auth0	Enterprise-grade auth platform	More expensive, more powerful than needed for current stage
Firebase Auth	Google's auth service	Would mean maintaining two different vendors (Google for auth, Supabase for data) with no data-layer integration benefit

Fresher-level insight: Since Supabase (the database) already includes free authentication, running Clerk *and* Supabase means paying for/maintaining two systems that both do "who is this user." This is a genuine area to evaluate — consolidating to Supabase Auth alone is a common cost-saving move once a product's login flows are simple enough.

4. Razorpay — Payments

What it is: Razorpay is a payment gateway — it lets the app accept money (cards, UPI, net banking, wallets) from users and handles the compliance/security of processing payments, since a small startup cannot legally or safely handle raw card numbers itself.

Why TradeSaath uses it:

- It's the leading payment gateway for **Indian businesses** — strong UPI support (India's dominant payment method), rupee-first pricing, and compliance with Indian regulations (RBI guidelines).
- Since TradeSaath is described as global-market but budget-conscious and built from India, Razorpay is the practical local choice for collecting revenue smoothly in INR.

Alternatives:

Alternative	What it is	Trade-off
Stripe	The global standard payment gateway	Excellent for international/global users, but historically has had more friction for Indian founders (business registration requirements, UPI support is weaker than Razorpay's)
PayPal	Widely known but higher fees, weaker India-specific rails	Not ideal as a primary gateway for an India-based fintech-adjacent product
Paddle / LemonSqueezy (Merchant of Record)	These platforms handle sales tax/VAT globally for you	Attractive if TradeSaath goes fully global — worth reconsidering once non-Indian customers grow, since they remove tax-compliance headaches Razorpay doesn't solve

Fresher-level insight: Since TradeSaath explicitly says it's a **global** product (not India-only), pure reliance on Razorpay is a mismatch worth flagging — a global user base will eventually need Stripe (or a Merchant-of-Record like Paddle) alongside or instead of Razorpay for smooth non-Indian payments.

5. Vercel — Hosting / Deployment

What it is: Vercel is the company that made Next.js, and their hosting platform is purpose-built to run Next.js apps. When code is pushed to GitHub, Vercel automatically builds and deploys it live on the internet — no manual server setup.

Why TradeSaath uses it:

- Zero-config deployment for Next.js — push code, get a live URL, including automatic "preview" URLs for testing changes before they go live.
- Handles global CDN (Content Delivery Network — serves your site fast from servers near each visitor worldwide) automatically.
- Free tier is enough for early-stage traffic.
- "Serverless Functions" — the API routes (backend code) run on-demand without managing a server 24/7.

Alternatives:

Alternative	What it is	Trade-off
Netlify	Similar deploy-from-GitHub platform	Slightly less optimized specifically for Next.js's newest features than Vercel (since Vercel builds Next.js itself)
AWS Amplify / raw AWS	Full control, deeply customizable	Much more complex to set up and maintain — overkill and time-costly for a solo founder
Railway / Render	Simpler alternative deploy platforms, often cheaper at scale	Less tightly integrated with Next.js-specific optimizations
Self-hosted VPS (e.g., DigitalOcean)	Rent a raw server and manage everything yourself	Cheapest at scale but requires real DevOps knowledge (security patches, uptime monitoring, scaling) — not worth it pre-revenue

Watch-out for a fresher: Vercel's free tier has **function timeout limits** (serverless functions can only run so many seconds) — this is a real, documented issue for things like AI API calls that take a while (a known "Claude API timeout on Vercel" problem class). Worth understanding early since it affects how AI coaching features are architected (e.g., streaming responses instead of waiting for one big response).

6. Anthropic Claude API — The AI Brain (Coaching, Analysis)

What it is: This is the AI model (Claude) that powers TradeSaath's psychology coaching, Decision Quality Score commentary, and session narratives — accessed via an API (a way for TradeSaath's code to "ask" the AI a question and get a response back), not a chat interface.

Why TradeSaath uses it:

- Claude is strong at nuanced, careful written analysis — a good fit for psychology/behavioral coaching text rather than just numeric prediction.
- Supports **prompt caching** (reusing a large, unchanging instruction block cheaply across many calls) — TradeSaath's coaching system prompt (~4,800 tokens) is cached, dramatically cutting cost per coaching call.
- Multiple model "sizes" (Haiku, Sonnet, Opus) let TradeSaath route cheap/simple tasks to a cheap model and complex reasoning to a more expensive one — a cost-control strategy already documented in the project.

Alternatives:

Alternative	What it is	Trade-off
OpenAI GPT models (via API)	The other major AI provider	Comparable quality; switching would mean rewriting prompts and losing Claude-specific caching setup already built
Google Gemini API	Google's AI models	Cheaper at some tiers, but less proven for the nuanced coaching-style writing TradeSaath needs
Open-source self-hosted models (e.g., Llama)	Run your own AI model on rented GPU servers	Would remove ongoing API costs but requires serious infra/ML-ops effort — not practical for a solo founder pre-scale

Cost-control idea already in use: TradeSaath replaced most **per-trade AI calls** with an algorithmic pattern detector (rules-based code, not AI), reserving AI calls for higher-value coaching moments — this is called out as a ~99% cost reduction. This is a genuinely smart pattern: **use AI only where its judgment is actually needed; use plain code/rules for everything mechanical.**

7. GitHub — Code Storage & Version Control

What it is: Git is a version-control system (tracks every change ever made to the code, like an infinite "undo history" with labeled checkpoints). GitHub is the website that hosts that history online, lets you collaborate, and connects to Vercel for auto-deployment.

Why TradeSaath uses it:

- Industry standard — virtually every tool (Vercel, CI/CD, AI coding assistants) integrates with GitHub by default.
- Free for private repositories at this scale.
- Every commit triggers a Vercel deployment automatically — code → GitHub → live site, all connected.

Alternatives:

Alternative	Trade-off
GitLab	Very similar, some prefer its built-in CI/CD; smaller ecosystem of AI tool integrations
Bitbucket	Common in Atlassian-heavy teams; less common for solo indie SaaS

There's little reason to switch off GitHub — it's the default choice with the least friction.

8. Resend — Transactional Email

What it is: Resend is a service for sending automated emails from code — signup confirmations, password resets, receipts, notifications — the emails your app sends *for* you, not marketing newsletters (necessarily).

Why TradeSaath uses it:

- Built specifically for developers (simple API, good deliverability — meaning emails are less likely to land in spam).
- Free tier covers early-stage email volume.
- Integrates cleanly with modern frameworks like Next.js.

Alternatives:

Alternative	Trade-off
SendGrid	Older, more enterprise-y, historically clunkier developer experience
AWS SES	Very cheap at scale but requires more manual setup (domain verification, sending limits raised manually)
Postmark	Excellent deliverability reputation, similar developer-first design to Resend — a close, valid alternative

9. Hostinger — Domain Registration

What it is: This is simply where the **domain name** (tradesaath.com) was purchased/registered — it doesn't host the actual website (Vercel does that); it just points the domain name to Vercel's servers.

Why TradeSaath uses it:

- Often cheaper domain pricing than bigger-name registrars, common choice for cost-conscious founders.

Alternatives:

Alternative	Trade-off
Namecheap	Very popular, competitive pricing, well-regarded for beginners
Google Domains (now Squarespace Domains)	Simple UI, slightly pricier
Cloudflare Registrar	Sells domains at-cost (no markup) — often the cheapest long-term option, plus free DNS/security tools bundled in

Fresher-level insight: Domain registrar choice barely matters technically (a domain is a domain) — the main things that matter are price and how easy the DNS settings panel is to use when connecting it to Vercel.

Overall Architecture — How It All Connects (Plain-English Flow)

1. **User visits tradesaath.com** → DNS (via Hostinger) points to **Vercel**, which serves the **Next.js** app.
 2. **User logs in** → **Clerk** handles the login screen and session.
 3. **User uploads/logs trades** → Next.js API routes save data into **Supabase** (Postgres database), protected by Row Level Security so users only see their own data.
 4. **Trade patterns get analyzed** → Mostly by TradeSaath's own algorithmic code (fast, free, reliable) with **Claude API** called in for nuanced psychology coaching text, using cached prompts to keep costs low.
 5. **User upgrades to a paid plan** → **Razorpay** processes the payment.
 6. **User gets a confirmation email** → **Resend** sends it.
 7. **Founder pushes a code update** → **GitHub** stores it, **Vercel** automatically rebuilds and redeploys the live site.
-

Where the Stack Could Genuinely Be Improved (Fresher-Friendly Summary)

Area	Current State	Possible Improvement	Why
SEO	Client-side rendering leaves Google seeing empty pages	Migrate key pages to Next.js Server Components (SSR)	Already identified as root cause of zero Google search visibility despite 129 indexed pages
Auth	Clerk <i>and</i> Supabase Auth both exist in the ecosystem	Evaluate consolidating to Supabase's built-in auth	Removes a whole paid service/dependency if Clerk's extra features aren't essential
Payments	Razorpay only, but product is positioned as global	Add Stripe or a Merchant-of-Record (Paddle/LemonSqueezy) for non-Indian users	Razorpay is India-optimized; a "global markets, all instruments" product will hit friction collecting international payments
AI cost control	Already using algorithmic detection + AI only for coaching, with prompt caching	Keep expanding this pattern — audit for any remaining per-action AI calls that could be rule-based instead	AI API calls are usually the single biggest variable cost in an AI SaaS product
Hosting limits	Vercel serverless function timeouts can affect long AI calls	Use streaming responses for AI coaching output (send text as it's generated instead of waiting for the full response)	Avoids timeout failures and feels faster to the user

Quick Glossary (for absolute beginners)

- **Frontend** = what the user sees and clicks (buttons, charts, forms).
- **Backend** = the behind-the-scenes code that saves data, checks passwords, talks to the database.
- **API** = a defined way for two pieces of software to talk to each other (e.g., your app "asking" Claude for a coaching message).
- **Database** = organized storage for all the app's data (users, trades, scores).

- **Deployment** = the process of taking code and making it live on the internet.
- **SSR (Server-Side Rendering)** = building a webpage's content on the server before sending it to the browser (good for speed and SEO).
- **Serverless Function** = a small piece of backend code that runs only when needed, instead of a server running 24/7.